

FabricCash

Frontend Execution Plan + Recommendation Engine Architecture

Textile CCC Optimisation Platform · Agent Source Material · v2.0

Upload-first flow · Free tier · AI-powered recommendations · Download results

7	5	6	2
Page sections	Max recommendations	Build agents	Rec. engine layers

0. Product Decisions Locked In

What changed from v1.0 — read before coding anything

Decision	What it means for the build
----------	-----------------------------

No paywall / pricing tiers	Everything is free. Remove all pricing sections, plan gating, and upgrade prompts from the codebase. The app is a public-good tool for the textile community. No Stripe integration needed in this phase.
Max 5 recommendations	The recommendation engine always returns a maximum of 5 ranked actions — never more. Sort by estimated impact (days freed). The top 5 are shown always, to all users equally.
No social proof / testimonials	Remove the testimonials section entirely. It doesn't add value in early launch. The page goes: Hero (upload) -> Tally export guide -> Live result -> How it works -> Footer.
Download results feature	After the CCC result is shown, a 'Download Report' button generates a PDF of the user's DIO, DSO, DPO, CCC score, benchmark comparison, and all 5 recommendations. Format: PDF (client-side via jsPDF). Optimal because: no server round-trip, instant, printable, and sharable with the broader team (not just the person who uploaded).
Target user: anyone with data	Not just CFOs. Anyone in the company who has access to the Tally/Excel export — accounts manager, finance intern, mill owner — can use this tool. Copy and UX must be accessible to non-finance users. Avoid jargon. Explain every metric inline (tooltip or inline definition).
App is fully client-side for the upload flow	File parsing (SheetJS), CCC calculation, and recommendation generation all happen in the browser for the landing page. No file is uploaded to any server. This is the privacy promise. Only when the user clicks 'Save results' does any data leave the browser (to Supabase via auth).

1. Recommendation Engine Architecture

Answering: do we use pure thresholds or an AI agent?

THE CORE QUESTION

Pure threshold rules (DIO > 45 days = flag it) are too rigid. A DIO of 52 days during Diwali season is completely normal for a cotton knit mill. The same DIO in March is a serious problem. Rigid rules produce irrelevant recommendations that the user learns to ignore. On the other hand, a pure LangChain agent with no structure is unpredictable and expensive. The answer is a two-layer hybrid architecture.

The Two-Layer Hybrid Architecture

The recommendation engine has two distinct layers that work in sequence. Layer 1 runs first and is deterministic. Layer 2 runs second and is AI-powered. Together they are fast, contextual, and explainable.

LAYER 1 — Rule Engine (deterministic, runs in browser)

A table of TypeScript rules. Each rule has: a condition function that receives the full CCCResult object, a base priority score (1-10), and a template function that generates the recommendation text. Rules are evaluated in milliseconds, entirely client-side, with no API call. This layer handles the structured, math-driven insights.

What it catches:

- DSO significantly above textile benchmark
- Specific invoices overdue by more than 60 days
- DPO well below what the industry typically negotiates
- DIO trending upward 3+ consecutive periods
- COGS data quality issues (estimated vs direct)
- Seasonal anomalies (flags suppressed in peak season)

Output: A pre-scored list of up to 8 candidate recommendations passed to Layer 2.

Layer 1 output feeds into Layer 2 →

LAYER 2 — Claude AI Contextualisation (runs server-side via Anthropic API)

Layer 1's candidate recommendations are passed as structured JSON to the Claude API (claude-sonnet-4-20250514). Claude's job is NOT to generate new recommendations from scratch. Its job is to: (1) re-rank them based on context it understands that rules cannot, (2) enrich each recommendation with textile-specific, human-readable explanation and action steps, and (3) suppress recommendations that are contradictory or contextually irrelevant. It returns the final top 5.

Why Claude, not LangChain?

LangChain adds abstraction complexity with no benefit here. You have one AI task: contextualise and re-rank a structured list. That is a single Claude API call with a well-crafted system prompt. No chains, no agents, no vector stores. LangChain is the right tool when you need multi-step tool-calling agents. Here you need one smart, fast inference call. Direct Anthropic API is simpler, cheaper, and easier to debug.

What Claude adds that rules cannot:

- Context-aware ranking: 'Your DSO issue matters more than DIO right now because 73% of your AR is concentrated in 2 buyers — fixing that has outsized impact'
- Seasonal intelligence: understands that October DIO spikes are pre-Diwali stocking
- Proportional severity: a 5-day DSO gap at Rs 2Cr revenue vs Rs 50Cr revenue is very different
- Human language: translates numeric gaps into plain business language any mill worker understands
- Contradiction detection: does not recommend both 'extend DPO' and 'pay early for discount' simultaneously

The API Route That Powers It

POST /api/recommendations

This is the only server-side call in the upload flow. Everything else (file parse, CCC math) runs in the browser. This route receives the CCCResult + Layer 1 candidates as JSON, calls Claude, and returns the final top 5 recommendations. Average latency: 1.5-3 seconds.

POST /api/recommendations — request body:

```
{ cccResult: { dio: { value: 52, trendDelta: +3, benchmark: 38 }, dso: { value: 67, trendDelta: +5, benchmark: 45 }, dpo: { value: 28, trendDelta: -2, benchmark: 42 }, ccc: 91, benchmarkCCC: 41, gapDays: 50 }, companyContext: { fabricTypes: ['cotton_knit', 'polyester_blend'], buyerTypes: ['retail_chain', 'exporter'], month: 10, // October – pre-Diwali context revenueRange: '10cr_50cr' }, layer1Candidates: [ // from rule engine { id: 'R1', dimension: 'DSO', priority: 9, title: '...', estimatedDays: 18 }, { id: 'R3', dimension: 'DIO', priority: 7, title: '...', estimatedDays: 12 }, ... up to 8 candidates ] }
```

The Claude system prompt (server-side, never exposed to client):

You are a working capital analyst specialising in Indian textile manufacturing. You receive a structured CCC analysis for a textile business and a list of candidate recommendations generated by a rule engine. Your tasks: 1. Re-rank the candidates by real-world impact given the company context (fabric types, buyer mix, revenue scale, and current month/season). 2. Suppress any recommendation that is contradictory with another higher-ranked one. 3. Enrich each of the top 5 with: a plain-language title (no jargon), a 2-sentence explanation, and 3 concrete action steps the user can take this week. 4. For each, state the estimated cash freed in rupees (based on the gap and revenue range). Return ONLY valid JSON. No preamble, no markdown. Schema: { recommendations: [{ id, dimension, priority, title, explanation, actionSteps: string[], estimatedDaysReduction, estimatedCashFreedLakhs }] }

Why This Architecture Is Better Than Pure Thresholds

Challenge	Problem with pure rules	Solution
Seasonal context	Rule: DIO > 45 days = alert. October DIO is 52 days. Pure rule fires. AI knows: October is pre-Diwali — cotton mills always stock up. It suppresses the alert or downgrades it.	AI layer handles this correctly
Revenue proportionality	Rs 5L overdue invoice at a Rs 5Cr mill = 10% of monthly revenue (critical). Same amount at Rs 100Cr mill = 0.5% (low priority). Rules cannot weigh this. AI re-ranks accordingly.	AI layer handles this correctly
Contradiction detection	Rules might fire both 'accept early payment discount' (implying you have cash) AND 'your cash position is tight' simultaneously. AI detects the contradiction and picks one.	AI layer handles this correctly

Plain language for non-finance users	The app serves anyone with the data — not just CFOs. A 'DPO gap of 14 days vs benchmark' means nothing to a mill owner. AI translates: 'You're paying suppliers 2 weeks faster than you need to — Rs 34L is leaving your account early every month.'	AI layer handles this correctly
Rule maintenance	Adding a new rule is one line in the rules table. The AI enrichment layer never needs to change when you add rules — it handles whatever Layer 1 sends it.	Clean separation of concerns

2. Landing Page Architecture

7 sections, upload-first, no paywall, no testimonials

S1

Navbar

Purpose	Logo (FabricCash + ■ icon) left. 'How it works' anchor link + 'Login' right. No pricing link. No sign-up button in nav — the hero IS the sign-up funnel.
Detail	Sticky on scroll. Transparent background that transitions to solid on scroll past hero. Height: 56px. Mobile: hamburger collapses to full-screen overlay.
Files	<code>components/Navbar.tsx</code>

S2

Hero + Upload Widget

Purpose	Headline: 'Find out in 60 seconds how much cash is trapped in your textile business.' Subheadline: 'Upload your Tally or Zoho Books export. Get your Cash Conversion Cycle instantly.' Upload widget is the dominant visual — not a button that leads to upload, but the widget itself. Privacy line below widget: 'Your file is processed in your browser. Nothing is uploaded.'
Detail	Drag-and-drop zone (dashed border, fabric texture background). Accepts .xlsx and .csv. Shows accepted software logos: Tally, Zoho Books, Busy, plain Excel. On file drop: shows spinner ('Parsing your data...') then smooth-scrolls to result section. Three file slots: Sales Register, Purchase Register, Stock Summary.
Files	<code>components/UploadWidget.tsx</code> + <code>lib/parser/sheetjs.ts</code>

S3

Tally Export Guide

Purpose	The single biggest friction point is users not knowing what to export. This section removes that friction before they feel it. Three-step visual guide with the exact Tally menu path for each of the 3 files.
Detail	Three cards side by side (stack on mobile). Each card: step number, file name, exact Tally path (monospace font), and a 'What we extract from this' tooltip. Below: 'Not on Tally?' link that downloads a pre-formatted Excel template.
Files	<code>components/TallyGuide.tsx</code> + <code>public/templates/fabriccash_template.xlsx</code>

S4	Live Result Panel
Purpose	Appears below upload widget after parsing completes (same page, smooth scroll). Hidden by default (opacity: 0, height: 0). Transitions in when results are ready. Shows: 4 metric cards (DIO, DSO, DPO, CCC), benchmark gap banner, top 5 recommendations, and Download Report button.
Detail	Metric cards: current value large, benchmark value small below, colour-coded (red if worse than benchmark, green if better). Gap banner: 'Rs X locked in working capital. Industry benchmark: Y days. Your gap: Z days.' Recommendations: priority-sorted cards with HIGH (red border), MEDIUM (amber), LOW (grey). Each shows: title, 2-line explanation, estimated days reduction, estimated cash freed. Download button: 'Download my CCC Report (PDF)'.
Files	<code>components/ResultPanel.tsx + lib/report/pdfGenerator.ts</code>

S5	How It Works
Purpose	Simple 3-step explainer for users who scrolled past the hero without uploading. Anchors to this from navbar 'How it works' link.
Detail	Three horizontal steps (stack on mobile): 1. Export from Tally (30 seconds) → 2. Upload 3 files → 3. Get your CCC + recommendations. Below steps: repeat the upload widget (same component, second instance). Headline: 'Still reading? Your cash is waiting.'
Files	<code>components/HowItWorks.tsx</code> (reuses <code>UploadWidget.tsx</code>)

S6	Sign Up Gate (modal / inline)
Purpose	Appears AFTER results are shown, when user clicks 'Save my results'. NOT a full page — a modal overlay or inline expansion below the result panel. Collects: email, password, company name, city, fabric types (checkboxes). On submit: creates Supabase auth user, saves CCCSnapshot to DB, closes modal, shows 'Saved!' confirmation. This is the only conversion point.
Detail	Simple form. Email + password + company name + city + fabric type checkboxes (Cotton knit / Polyester blend / Technical textiles / Yarn / Fabric trading). Privacy line: 'We store only your aggregated CCC metrics, not your raw invoice data.' No credit card. No plan selection.
Files	<code>components/SignUpModal.tsx + lib/auth/supabase.ts</code>

S7	Footer
Purpose	Minimal. Brand left. Links right. No newsletter. No social icons.
Detail	'Built for textile mills across India — Surat, Tiruppur, Bhilwara, Panipat.' Links: Privacy Policy, Terms of Use, Contact (mailto link). Copyright line.
Files	<code>components/Footer.tsx</code>

3. User Flow — Step by Step

Exact sequence of events from first visit to saved results

1	Arrives on page	User sees: navbar + hero headline + upload widget with 3 file slots. No modals, no popups, no cookie banners blocking the widget.
2	Reads Tally guide (optional)	If confused about what to upload, scrolls slightly to see the 3-step Tally export guide. Downloads Excel template if not on Tally.
3	Drops 3 files into widget	Drags Sales Register.xlsx, Purchase Register.xlsx, Stock Summary.xlsx into the 3 slots. Or clicks each slot to open file picker. Widget validates file type immediately.
4	Parsing spinner shows	Widget shows 'Parsing your data...' with a subtle progress indicator. SheetJS runs entirely in browser. Typical parse time: 200-800ms for a 500-row file.
5	Page scrolls to result panel	On parse complete, page smooth-scrolls to the result panel. Metrics animate in (count-up animation on DIO, DSO, DPO, CCC values). POST <code>/api/recommendations</code> is called simultaneously (AI enrichment, ~2s).
6	Recommendations load	While metrics are visible immediately (from Layer 1 client-side rules), the AI-enriched recommendations replace the skeleton cards when the API returns. User sees: 5 ranked cards with titles, explanations, action steps, and cash impact.
7	User downloads report (optional)	Clicks 'Download my CCC Report'. jsPDF generates a PDF client-side in ~1 second. PDF contains: company context, 4 metric cards with benchmark comparison, all 5 recommendations with action steps, and a 'Generated by FabricCash' footer. File saves as 'FabricCash_CCC_Report_[date].pdf'.
8	User clicks 'Save results'	Sign-up modal appears over the result panel (result stays visible in background). User fills: email, password, company name, city, fabric types.
9	Account created, results saved	Supabase creates the auth user. A CCCSnapshot row is written to the database. Modal closes. A subtle toast appears: 'Results saved. You can track your CCC over time.' User is NOT redirected away from the page — they stay on the result. Dashboard link appears in navbar for future visits.

4. File Parsing Specification

What we extract from each Tally export and how we map it

ACCEPTED FORMATS

Primary: .xlsx (Excel) — exported from Tally Prime, Zoho Books, Busy Accounting, or hand-filled template. Fallback: .csv (comma-separated). Parser: SheetJS (xlsx library, browser-compatible). Max file size: 10MB per file (enough for 5 years of a Rs 50Cr mill's transactions). Parse happens 100% in browser — no file touches the server.

File	What we parse	Engine output
Sales Register (AR Invoices)	Tally path: Gateway -> Display -> Account Books -> Sales Register -> Alt+E -> Excel Columns we look for: Party Name, Date, Invoice No, Gross Total, Due Date What we extract: Customer name, invoice date, amount, due date What we compute: Whether payment is received (if Payment Date column exists)	-> AR invoice records -> Used for DSO calculation -> paymentDate null if not paid
Purchase Register (AP Invoices)	Tally path: Gateway -> Display -> Account Books -> Purchase Register -> Alt+E -> Excel Columns we look for: Party Name, Date, Invoice No, Gross Total, Due Date What we extract: Vendor name, invoice date, amount, due date, payment date Key field: paymentDate — if null, invoice is still outstanding	-> AP invoice records -> Used for DPO calculation -> Outstanding = null paymentDate
Stock Summary (Inventory)	Tally path: Gateway -> Display -> Inventory Books -> Stock Summary -> Alt+E -> Excel Columns we look for: Stock Item, Closing Balance (Qty), Rate, Closing Value What we extract: Item name, closing stock value (in Rs), period Grouped by fabric type if item names contain keywords (cotton, polyester, etc.)	-> InventoryItem records -> Used for DIO calculation -> COGS derived from Sales if absent

Column Name Fuzzy Matching Strategy

Tally exports have slightly different column headers depending on version and configuration. The parser uses a fuzzy matching approach: for each expected field, it checks a list of known aliases. If no alias matches, the user is shown an interactive column mapper ('Which column is the invoice date?').

Accepted column name aliases (any of these)	Maps to field
Date / Invoice Date / Voucher Date / Date of Invoice	invoiceDate
Party Name / Party's Name / Customer / Vendor / Supplier Name	counterpartyName

Gross Total / Amount / Net Amount / Total / Bill Amount	amount
Due Date / Payment Due / Due On / Credit Period End	dueDate
Closing Value / Stock Value / Inventory Value / Balance Value	inventoryValue

5. Download Report Feature

jsPDF client-side PDF generation — format, content, and architecture

WHY PDF (CLIENT-SIDE jsPDF)

Format options considered: CSV (too raw, not shareable), Excel (requires server), PNG screenshot (not text-searchable), Server-side PDF (adds latency + server cost). Winner: jsPDF in the browser. Generates in ~1 second, no server round-trip, produces a professional printable document the user can email to their accountant or mill owner. Library: jsPDF + jspdf-autotable for the recommendation table.

PDF Report Contents

Section	Content
Page 1 — Header	FabricCash logo + 'Cash Conversion Cycle Report'. Company name (if signed in) or 'Guest Analysis'. Analysis date. Period analysed (30/60/90 days). Data source (Tally / Excel template).
Page 1 — CCC Score Panel	4 metric cards in a 2x2 grid: DIO, DSO, DPO, CCC. Each shows: current value (large), textile industry benchmark (small), gap (+ or - days, colour coded), trend vs last period.
Page 1 — Benchmark Banner	Highlighted box: 'Your CCC is X days. Textile industry benchmark: Y days. Estimated working capital locked: Rs Z lakhs.' Red if CCC > benchmark, green if below.
Page 1 — Top 5 Recommendations	Table with 5 rows. Columns: Priority (HIGH/MEDIUM/LOW badge), Dimension (DSO/DIO/DPO), Recommendation title, Action steps (numbered), Estimated impact (days + Rs lakhs freed).
Page 2 — Methodology note	One paragraph explaining CCC = DIO + DSO - DPO. Definitions of each metric in plain language. Data quality note (if COGS was estimated, flag it here). Benchmark source: 'Industry benchmarks from SIDBI SME Report 2023-24'.
Footer (every page)	'Generated by FabricCash · fabriccash.in · Analysis date: [date]'. Disclaimer: 'This report is for internal planning purposes only.'

Implementation in lib/report/pdfGenerator.ts

```
import jsPDF from 'jspdf' import autoTable from 'jspdf-autotable' export function
generateCCCReport(result: CCCResult, context: CompanyContext): void { const doc = new jsPDF({
orientation: 'portrait', unit: 'mm', format: 'a4' }) // Page 1: header, metric cards, benchmark banner
drawHeader(doc, context) drawMetricCards(doc, result) drawBenchmarkBanner(doc, result) // Recommendations
table autoTable(doc, { startY: 140, head: [['Priority', 'Dimension', 'Recommendation', 'Impact']], body:
result.recommendations.map(r => [ r.priority, r.dimension, r.title + '\n' + r.actionSteps.join(' | '),
`-${r.estimatedDaysReduction}d / Rs ${r.estimatedCashFreedLakhs}L` ]), styles: { fontSize: 8,
cellPadding: 4 }, headStyles: { fillColor: [27, 58, 107] } // INDIGO }) // Page 2: methodology
doc.addPage() drawMethodology(doc) // Save doc.save(`FabricCash_CCC_Report_${new
Date().toISOString().split('T')[0]}.pdf`) }
```


6. Agent Build Plan — 6 Agents in Sequence

Feed each card to your IDE agent in order. Each output feeds the next.

Agent 1	Project scaffold
Owns	Bootstrap the Next.js project with all config, folder structure, and local Docker setup. No business logic.
Files	<code>docker-compose.yml</code> , <code>.env.local</code> , <code>.env.example</code> , <code>/lib/*</code> (empty), <code>/components/*</code> (empty)
Prompt	<i>"Create Next.js 14 App Router project with TypeScript strict mode, Tailwind CSS, shadcn/ui. Add Prisma + PostgreSQL. Create docker-compose.yml with Postgres 15. Create .env.local with DATABASE_URL pointing to local Postgres. Add package.json scripts: db:up, db:seed, dev, db:studio. Create empty folders: /lib/ccc-engine, /lib/parser, /lib/report, /lib/recommendations, /lib/auth, /components/landing, /components/dashboard."</i>

Agent 2	SheetJS file parser
Owns	Build the browser-side Excel/CSV parser that converts Tally exports into typed invoice, inventory, and sales records.
Files	<code>/lib/parser/sheetjs.ts</code> , <code>/lib/parser/types.ts</code> , <code>/lib/parser/columnAliases.ts</code>
Prompt	<i>"Using SheetJS (xlsx library), build /lib/parser/sheetjs.ts. It must: accept a File object, detect whether it is Sales Register, Purchase Register, or Stock Summary by sniffing column headers. Apply fuzzy column matching (the alias table from Section 4 of the blueprint). Return typed ParsedInvoice[], ParsedInventoryItem[], ParsedSalesRecord[]. Handle missing columns gracefully — return { data, warnings: string[] }. Write a /lib/parser/types.ts with all parsed types. Test with a mock Tally export."</i>

Agent 3	CCC engine + Layer 1 rules
Owns	Build the pure TypeScript CCC calculator and the Layer 1 rule engine. Both run entirely client-side with no API calls.
Files	<code>/lib/ccc-engine/calculator.ts</code> , <code>/lib/ccc-engine/types.ts</code> , <code>/lib/ccc-engine/benchmarks.json</code> , <code>/lib/recommendations/layer1Rules.ts</code>
Prompt	<i>"Build /lib/ccc-engine/calculator.ts with pure functions: calculateDIO(), calculateDSO(), calculateDPO(), calculateCCC(). Use types from the blueprint Section 6. Then build /lib/recommendations/layer1Rules.ts — a typed array of RecommendationRule objects (the 8 rules from the blueprint Section 7). Export evaluateLayer1(cccResult): Candidate[] that filters rules by condition and returns scored candidates. Add /lib/ccc-engine/benchmarks.json with textile industry DIO/DSO/DPO benchmarks."</i>

Agent 4	API route — recommendations
---------	-----------------------------

Owns	Build the single server-side API route that calls Claude for Layer 2 AI enrichment.
Files	/app/api/recommendations/route.ts, /lib/recommendations/claudeClient.ts
Prompt	<i>"Build /app/api/recommendations/route.ts as a Next.js POST route handler. It receives { cccResult, companyContext, layer1Candidates } as JSON (Zod-validated). It calls the Anthropic API (claude-sonnet-4-20250514) with the system prompt from Section 1 of this blueprint. It parses the JSON response and returns the top 5 enriched recommendations. Handle API errors gracefully — if Claude fails, fall back to returning Layer 1 candidates with template descriptions (never show an error to the user). Add ANTHROPIC_API_KEY to .env.local."</i>

Agent 5	Landing page + upload widget
Owns	Build the full landing page with all 7 sections and the upload widget as the centrepiece.
Files	/app/page.tsx, /components/landing/UploadWidget.tsx, /components/landing/ResultPanel.tsx, /components/landing/TallyGuide.tsx, /lib/report/pdfGenerator.ts
Prompt	<i>"Build /app/page.tsx as the landing page. Build /components/landing/UploadWidget.tsx — drag-and-drop zone with 3 file slots (Sales Register, Purchase Register, Stock Summary). On file drop: call the SheetJS parser, then calculateCCC(), then call POST /api/recommendations. Build /components/landing/ResultPanel.tsx — hidden by default, smooth-scrolls into view, shows 4 metric cards + benchmark banner + 5 recommendation cards + Download Report button. Build /components/landing/TallyGuide.tsx — 3-step export instructions. Build /components/landing/HowItWorks.tsx. Wire the Download Report button to jsPDF (install jspdf + jspdf-autotable). Build /lib/report/pdfGenerator.ts per Section 5."</i>

Agent 6	Auth + save results
Owns	Add Supabase auth, the sign-up modal, and the DB write when a user saves their results.
Files	/lib/auth/supabase.ts, /components/landing/SignUpModal.tsx, /app/api/snapshots/save/route.ts, /app/login/page.tsx, prisma/schema.prisma
Prompt	<i>"Install @supabase/ssr and configure Supabase client in /lib/auth/supabase.ts. Build Prisma schema with Company, CCCSnapshot models. Run prisma migrate dev. Build /components/landing/SignUpModal.tsx — email, password, company name, city, fabric type checkboxes. On submit: create Supabase auth user, then POST to /api/snapshots/save with the CCCResult + company details. Build that API route. Show a toast on success. Add Login page at /app/login/page.tsx. Update Navbar to show 'Dashboard' link when user is authenticated."</i>

7. Complete File Structure

```
/app/ ■■■■ page.tsx ← Landing page (Agent 5) ■■■■ login/page.tsx ← Login page (Agent 6) ■■■■
dashboard/page.tsx ← CCC history dashboard (post-MVP) ■■■■ api/ ■■■■ recommendations/route.ts ← Claude
Layer 2 call (Agent 4) ■■■■ snapshots/save/route.ts ← Save CCCSnapshot (Agent 6) /components/ ■■■■
landing/ ■ ■■■■ Navbar.tsx ← Agent 5 ■ ■■■■ UploadWidget.tsx ← Agent 5 (core component) ■ ■■■■
TallyGuide.tsx ← Agent 5 ■ ■■■■ ResultPanel.tsx ← Agent 5 ■ ■■■■ MetricCard.tsx ← Agent 5 ■ ■■■■
RecommendationCard.tsx ← Agent 5 ■ ■■■■ HowItWorks.tsx ← Agent 5 ■ ■■■■ SignUpModal.tsx ← Agent 6 ■
■■■■ Footer.tsx ← Agent 5 ■■■■ ui/ ← shadcn/ui components /lib/ ■■■■ parser/ ■ ■■■■ sheetjs.ts ← Agent 2
(file parsing) ■ ■■■■ types.ts ← Agent 2 ■ ■■■■ columnAliases.ts ← Agent 2 (fuzzy matching) ■■■■
ccc-engine/ ■ ■■■■ types.ts ← Agent 3 (source of truth) ■ ■■■■ calculator.ts ← Agent 3 ■ ■■■■
benchmarks.json ← Agent 3 (textile benchmarks) ■■■■ recommendations/ ■ ■■■■ layer1Rules.ts ← Agent 3
(rule table) ■ ■■■■ claudeClient.ts ← Agent 4 ■■■■ report/ ■ ■■■■ pdfGenerator.ts ← Agent 5 (jsPDF) ■■■■
auth/ ■■■■ supabase.ts ← Agent 6 /prisma/ ■■■■ schema.prisma ← Agent 6 ■■■■ seed.ts ← Agent 6 /public/
■■■■ templates/ ■■■■ fabriccash_template.xlsx ← Hand-crafted (not agent) .env.local ← Agent 1
.env.example ← Agent 1 docker-compose.yml ← Agent 1
```